

Post-Quantum Cryptography on Small RISC-V Processors: Benchmark Report

Oct 6, 2026 · @SSMind5 Technologies

Executive summary

All three finalized NIST post-quantum standards run correctly on a minimal 32-bit RISC-V core (RV32IM), but they differ sharply in cost, and the right choice depends on the device. We measured ML-KEM, ML-DSA and SLH-DSA with exact instruction counts, verified every result, and reproduced the benchmark on two machines with two compiler generations.

- **ML-KEM is practical even on small cores:** 1.0–3.1 million instructions and 6–20 kB of stack per operation.
- **ML-DSA signing time is unpredictable:** the slowest signature in 15 runs cost up to 2.7 times the median, so devices with deadlines must budget for the worst case.
- **ML-DSA's portable code is stack-hungry:** 52–122 kB, half of a 256 KB microcontroller at the highest security level.
- **SLH-DSA is the low-memory option:** about 4 kB of stack, but slow to sign. On 32-bit cores its SHA-2 variants are 4.6 times cheaper than SHAKE.
- **One compiler flag cuts ML-KEM's cost by 23–29%,** with no code changes, confirmed on GCC 10.2 and GCC 13.2.

Why this matters

Devices shipping today will still be in service when quantum-vulnerable cryptography is retired. India's national roadmap and NIST's transition guidance both target migration around 2030–2035, while smart meters, payment terminals and industrial controllers typically stay in the field for 10–20 years.

Small RISC-V processors are where this is hardest. They have limited memory and no spare performance, and the new algorithms are heavier than the RSA and elliptic-curve cryptography they replace. Before committing to a design, device makers need three answers: which algorithm fits, how much memory and time it costs, and what can be tuned. This report gives measured answers for a minimal RV32IM core.

Method

We used `rvpqc`, our open-source benchmarking tool, on vetted portable implementations of the three standards, run bare-metal on an emulated RV32IM core.

Item	Detail
Algorithms	ML-KEM-512/768/1024 (FIPS 203), ML-DSA-44/65/87 (FIPS 204), SLH-DSA-SHA2-128f and SLH-DSA-SHAKE-128f (FIPS 205)

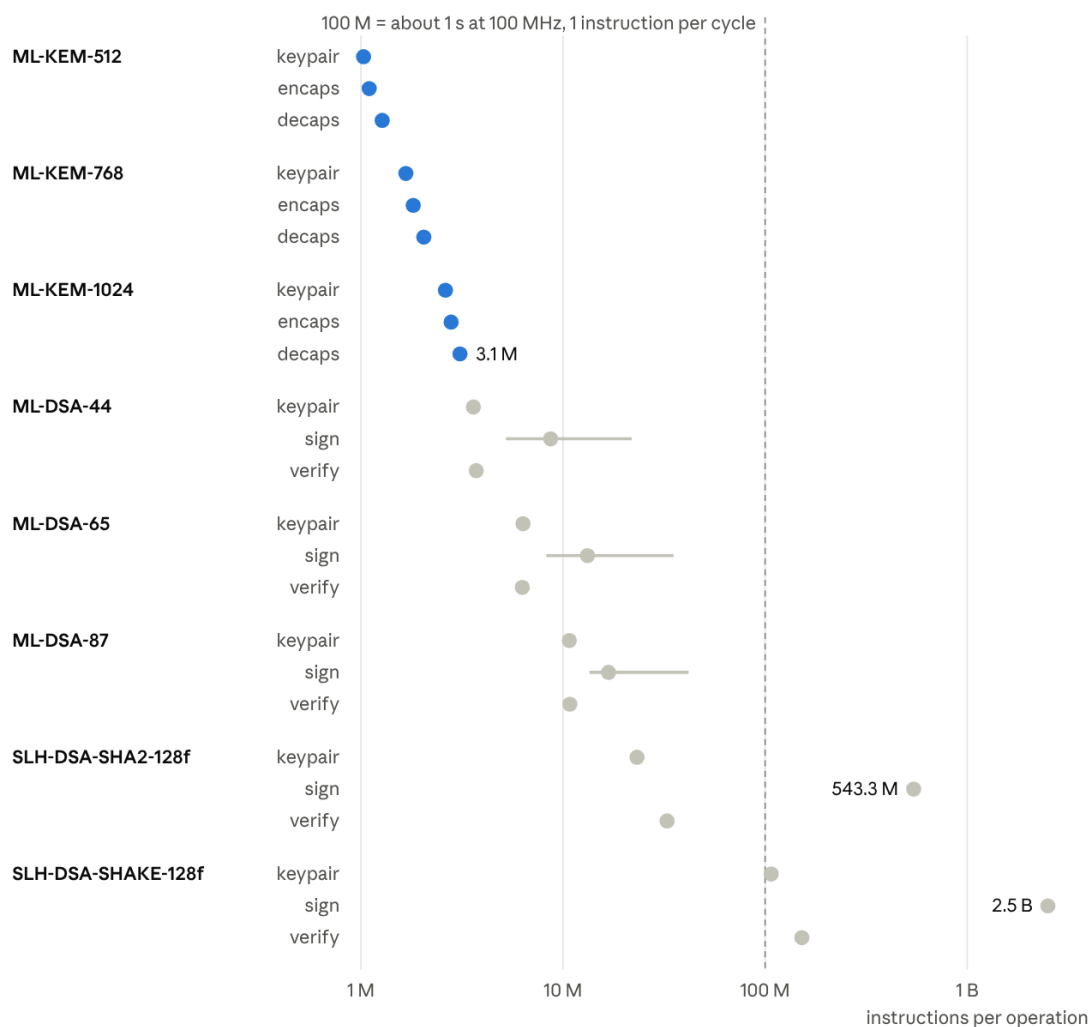
Item	Detail
Implementations	PQClean clean code for ML-KEM and ML-DSA; slhdsa-c from the PQ Code Package for SLH-DSA
Platform	QEMU virt board, RV32IM, no operating system
Instruction counts	64-bit instret counter with QEMU -icount shift=0, which makes it exact; measurement overhead calibrated and subtracted
Stack	Stack filled with a known pattern before each operation; deepest overwritten point measured after
Code size	.text of the algorithm and its hash code, excluding the test harness
Runs	15 per operation for ML-KEM and ML-DSA, 2 for SLH-DSA (its cost barely varies)
Main toolchain	GCC 13.2, -O2; QEMU 8.2
Reproduction	Second machine: GCC 10.2, QEMU 6.2

Every result was checked three ways. Key-exchange secrets had to match and signatures had to verify, while tampered signatures had to be rejected. Every key, ciphertext and signature had to be byte-identical to a native build of the same code with the same deterministic inputs. A built-in self-test plants a one-bit error to prove that comparison catches it.

Results

Key exchange costs about 1 to 3 million instructions; signing costs up to 2.5 billion

Median instructions per operation on RV32IM, log scale; whiskers show the fastest and slowest run



rvpqc on QEMU, RV32IM, GCC 13.2 -O2 · 15 runs per operation (SLH-DSA: 2) · October 2026

Every ML-KEM operation stays near or below 3 million instructions, while SLH-DSA signing crosses the 100-million line that marks about a second on a 100 MHz core. Memory and code size tell a different story:

Algorithm	Peak stack (kB)	Code size (kB)
ML-KEM-512	9.4	23.1
ML-KEM-768	14.4	23.3
ML-KEM-1024	20.5	24.1

Algorithm	Peak stack (kB)	Code size (kB)
ML-DSA-44	51.6	30.6
ML-DSA-65	79.3	30.2
ML-DSA-87	122.3	30.3
SLH-DSA-SHA2-128f	4.0	52.4*
SLH-DSA-SHAKE-128f	3.7	52.4*

Peak stack is the largest of the three operations. kB = 1,000 bytes. *SLH-DSA's code size covers the whole slhdsa-c library (all 12 parameter sets), so it is the same for both variants.

Findings in detail

ML-KEM fits small devices

ML-KEM-768, the level most deployments choose, needs 1.7–2.0 million instructions per operation and at most 14.4 kB of stack. At an illustrative 1 instruction per cycle and 100 MHz, that is about 20 ms per operation; real cores need more cycles per instruction, so this is a lower bound.

ML-DSA signing time varies, and its worst case matters

ML-DSA signing retries internally until it produces a valid signature, so its cost varies from run to run. Verification does not.

Parameter set	Sign, median	Sign, worst of 15	Worst ÷ median
ML-DSA-44	8.7 M	21.9 M	2.5×
ML-DSA-65	13.2 M	35.2 M	2.7×
ML-DSA-87	16.8 M	41.8 M	2.5×

A device with a signing deadline must be sized for the worst case, not the average. Fifteen runs only sample the distribution, so true worst cases can be higher still.

ML-DSA's portable code needs a lot of stack

Peak stack ranges from 52 kB (ML-DSA-44) to 122 kB (ML-DSA-87). On a microcontroller with 256 kB of SRAM, ML-DSA-87 would take nearly half the memory as stack alone. Stack-optimized implementations exist and can reduce this substantially.

SLH-DSA trades speed for a tiny memory footprint

SLH-DSA needs only about 4 kB of stack, but signing is slow: 543 million instructions for SLH-DSA-SHA2-128f. On a 32-bit core, choose its SHA-2 variants: SHAKE-128f costs 4.6 times as much to sign, because SHAKE's Keccak works on 64-bit words that a 32-bit core must emulate.

The "s" variants suit firmware signing. In a separate run, SLH-DSA-SHA2-128s signed about 21 times slower than SHA2-128f (11.3 billion instructions) but verified about 3 times faster, with signatures less than half the size (7,856 vs 17,088 bytes). Firmware is signed once in the factory and verified at every boot.

One compiler flag cuts ML-KEM's cost by about a quarter

With GCC's default `-O2`, instruction scheduling before register allocation makes the Keccak permutation spill heavily to the stack on RV32. Adding `-fno-schedule-insns` removes most of those spills.

ML-KEM-768	GCC 13.2	GCC 10.2
Encapsulation, <code>-O2</code>	1,814,448	1,758,102
Encapsulation, <code>-O2 -fno-schedule-insns</code>	1,312,840 (−27.6%)	1,296,149 (−26.3%)
Encapsulation, <code>-Os</code>	1,427,933	—

On GCC 10.2 the same flag also cut key generation by 28.7%, decapsulation by 23.3%, and code size by 14%. No source code changes are needed.

Code size is tunable too

Compressed instructions (RV32IMC) shrink ML-KEM-768's code from 23,348 to 17,020 bytes, and combined with `-Os` to 12,808 bytes (−45%), at essentially no instruction-count cost for compressed instructions alone.

Recommendations for device makers

1. **Key exchange: start with ML-KEM-768.** It fits comfortably on small cores; use ML-KEM-512 only where memory or time is truly tight and the security level is acceptable.
2. **Signatures on memory-constrained devices: prefer SLH-DSA with SHA-2,** especially for verifying firmware. Its 4 kB stack fits almost anywhere; choose the "s" variants when signing happens offline.
3. **If you use ML-DSA, plan for stack and worst-case time.** Budget 52–122 kB of stack for portable code, or adopt a stack-optimized implementation, and size deadlines for at least 2.5–3 times the median signing cost.
4. **Tune the build before optimizing code.** Test `-fno-schedule-insns`, `-Os` and compressed instructions on your exact toolchain; together they cut cost by up to a quarter and code by up to 45% in our measurements.
5. **Measure on your target.** Instruction counts guide design choices, but final timing must come from the real core, with its pipeline and memory system.

Limitations and next steps

These results are instruction counts on an emulated generic core, not cycle counts on a specific chip.

- **Cycles differ from instructions.** On small in-order cores, multiplies, loads and taken branches often take more than one cycle.

- **Portable code only.** Optimized implementations, or custom instructions, can be much faster.
- **Simple runtime.** The harness uses byte-wise memory copies and a basic allocator, so counts are slightly conservative.
- **Toolchain-dependent.** Exact counts shift by 1–3% between compiler versions; compare only results from the same toolchain.

Next, we plan to measure real cycle counts on RISC-V development boards, including India's indigenous VEGA processors, add stack-optimized and speed-optimized implementations, and cover Falcon/FN-DSA and HQC once standardized.

About

This benchmark was produced with **rvpqc**, an open-source post-quantum performance lab for RISC-V developed by SSMIND5 Technologies and released under the MIT license. Anyone can reproduce every number in this report with one command: github.com/ssmind5tech/rvpqc.

SSMIND5 Technologies helps device makers and chip teams adopt post-quantum cryptography: feasibility studies on your exact processor, algorithm and configuration selection, and optimization of memory, time and code size.

CONTACT: info@ssmind5tech.in; ssmind5techpvtltd@gmail.com